
API SECURITY RISK ANALYSIS REPORT

Cybersecurity Task 3 - Future Interns

Organisation	Future Interns
Prepared by	Muhammad Ali
Assessment Approach	Read-Only API Security Risk Analysis
Target API	ReqRes API (reqres.in)

Table of Contents

Executive Summary	1
Scope & Methodology.....	2
Scope	2.1
Methodology.....	2.2
Tools Used.....	2.3
API Security Findings	3
API Security Assessment Findings Overview	4
Remediation Recommendations	5
Conclusion	6

1. Executive Summary

This report outlines the results of a read-only API security risk analysis conducted on the publicly accessible ReqRes API. The assessment was performed using passive testing methods with Insomnia and Browser Developer Tools to assess authentication controls, response headers, security configurations, and browser-based access mechanisms.

The analysis identified five key observations, including authentication enforcement, information disclosure through response headers, implementation of HTTP security headers, access control validation, and a permissive Cross-Origin Resource Sharing (CORS) configuration. Most observations were classified as low or informational, while some represented positive security controls already implemented by the platform.

Overall, no critical vulnerabilities, authentication bypasses, or unauthorised data exposure were detected during the assessment. The API demonstrated a relatively strong security posture and effective implementation of fundamental security controls. Based on the findings identified, the overall security risk of the API was assessed as low. Recommendations have been provided to further strengthen security and support ongoing compliance with OWASP API security best practices.

2. Scope & Methodology

2.1 Scope

The scope of this assessment was restricted to passive security testing of the publicly accessible ReqRes API. The analysis covered endpoint inspection, authentication review, HTTP header inspection, response analysis, and safe input validation testing using read-only GET requests and authorised POST requests. The assessment excluded exploitation attempts, denial-of-service testing, authentication bypassing, and testing against private or production systems.

2.2 Methodology

The assessment was carried out using a structured passive testing methodology. The public API documentation was first analysed to understand accessible endpoints and supported request methods. API requests were then tested using Insomnia and browser developer tools to inspect responses, headers, authentication behaviour, and exposed data. The gathered information was evaluated against common API security risks, including weak authentication, excessive data exposure, missing security headers, insufficient rate limiting, and input validation weaknesses. All procedures were performed within ethical and legal boundaries.

2.3 Tools Used

Tool	Purpose
Insomnia	Used for API request testing and endpoint analysis
Browser Developer Tools	Used to inspect headers, network requests, and response behaviour

3. API Security Findings

Finding 1 - Unauthorised Request Blocking

Description

A security assessment was performed on the ReqRes API endpoint.

GET <https://reqres.in/api/users?page=1>

The request was sent without providing a valid API authentication key. The API correctly denied access and returned an HTTP 401 unauthorised response. The response body explicitly indicated that the x-api-key header was needed before access could be granted.

The analysis shows that the API implements authentication controls to prevent unauthorised access to secured resources. During testing, Browser Developer Tools and Insomnia were used to inspect request and response headers, response codes, and authentication behaviour.

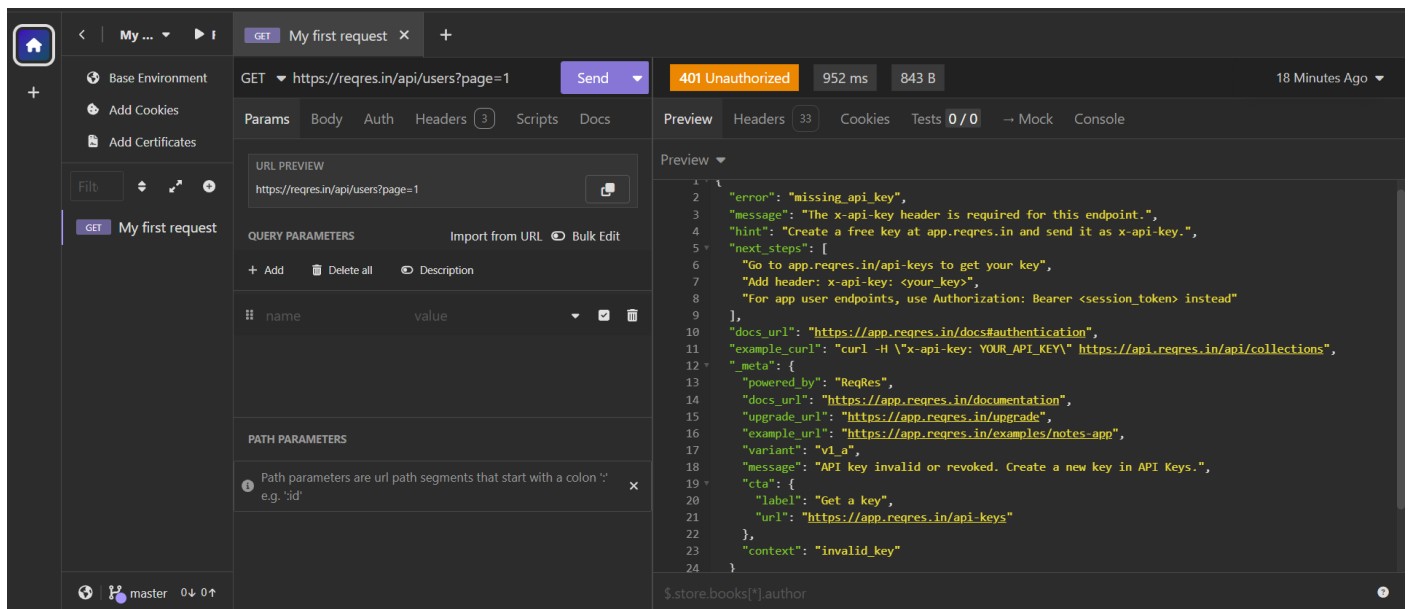


Figure 1: Unauthorised API request to the ReqRes endpoint returning HTTP 401 due to missing API authentication

Preview

Headers33

Cookies

Tests0 / 0

→ Mock

Console

NAME	VALUE
date	Tue, 26 May 2026 21:40:39 GMT
content-type	application/json; charset=utf-8
content-length	843
access-control-allow-origin	*
cache-control	no-store
cross-origin-opener-policy	same-origin
cross-origin-resource-policy	same-origin
etag	W/"34b-ktCrb7cpZ6PqSO/9y+g1JrA9MI0"
nel	{"report_to":"heroku-nel","response_headers":["Via"],"max_age":3600,"success_fraction":0.01,"failure_fraction":0.1}
origin-agent-cluster	?1
referrer-policy	strict-origin-when-cross-origin
report-to	{"group":"heroku-nel","endpoints":[{"url":"https://nel.heroku.com/reports?s=5D%2B9F33IFihROU9DeXs3NomhiBbjvTc%2BP3mVDLthuC4%3D\u0026sid=c4c9725f-1ab0-44d8-820f-430df2718e11\u0026ts=1779831639"}],"max_age":3600}

Figure 2: Initial response headers returned by the API after the unauthorised request

Preview	Headers	33	Cookies	Tests	0 / 0	→ Mock	Console
referrer-policy		strict-origin-when-cross-origin					
report-to		{ "group": "heroku-nel", "endpoints": [{ "url": "https://nel.heroku.com/reports?s=5D%2B9F33IFihROU9DeXs3NomhiBbjvTc%2BP3mVDLthuC4%3D\u0026sid=c4c9725f-1ab0-44d8-820f-430df2718e11\u0026ts=1779831639"}], "max_age": 3600 }					
reporting-endpoints		heroku-nel="https://nel.heroku.com/reports?s=5D%2B9F33IFihROU9DeXs3NomhiBbjvTc%2BP3mVDLthuC4%3D&sid=c4c9725f-1ab0-44d8-820f-430df2718e11&ts=1779831639"					
server		cloudflare					
strict-transport-security		max-age=31536000; includeSubDomains					
via		1.1 heroku-router					
x-content-type-options		nosniff					
x-dns-prefetch-control		off					
x-download-options		noopen					
x-frame-options		DENY					

Figure 3: Additional response header information observed during the API analysis

Preview

Headers33

Cookies

Tests0 / 0

→ Mock

Console

x-frame-options	DENY
x-permitted-cross-domain-policies	none
x-powered-by	ReqRes.in - Deploy backends in 30 seconds
x-reqres-docs	https://reqres.in
x-reqres-help	missing_api_key
x-reqres-message	This API is powered by ReqRes. Deploy your own backend in 30 seconds!
x-reqres-templates	https://app.reqres.in/templates
x-reqres-tip	Create your own backend with persistent data - app.reqres.in/dashboard
x-reqres-upgrade	https://app.reqres.in/upgrade
x-request-id	1438ed57-ad4b-a2fe-8dd6-dab4f8f8481a
x-robots-tag	noindex, nofollow
x-xss-protection	0
cf-cache-status	BYPASS

Figure 4: Security-related HTTP headers identified during response inspection

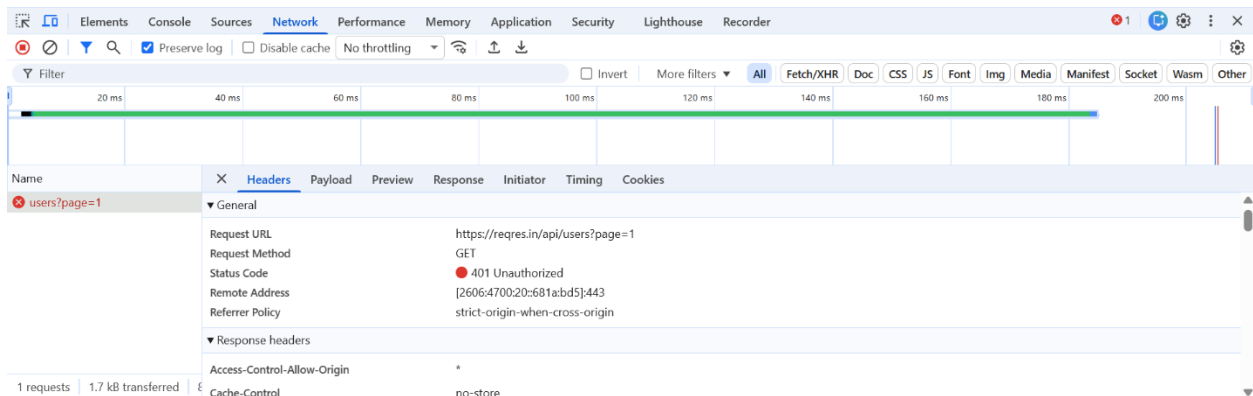


Figure 5: Chrome DevTools network inspection confirming the unauthorised API response

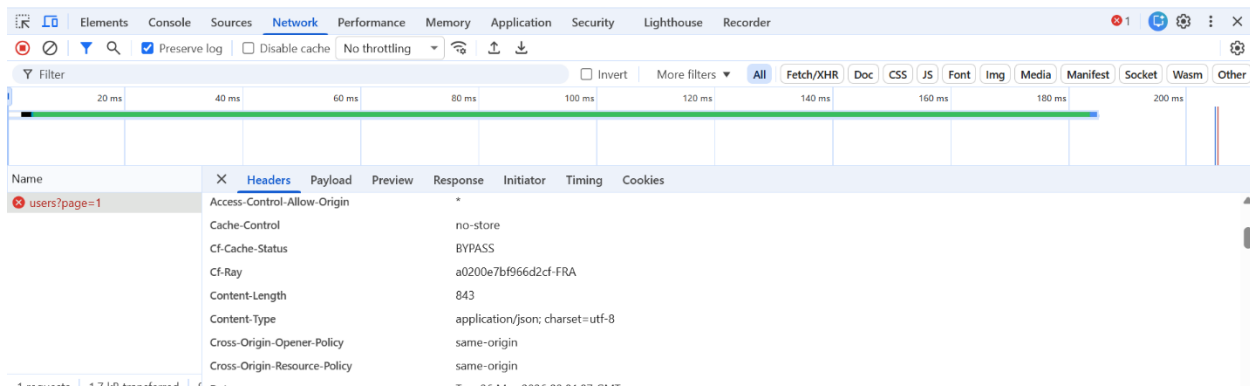


Figure 6: Response header validation showing caching and security configuration details

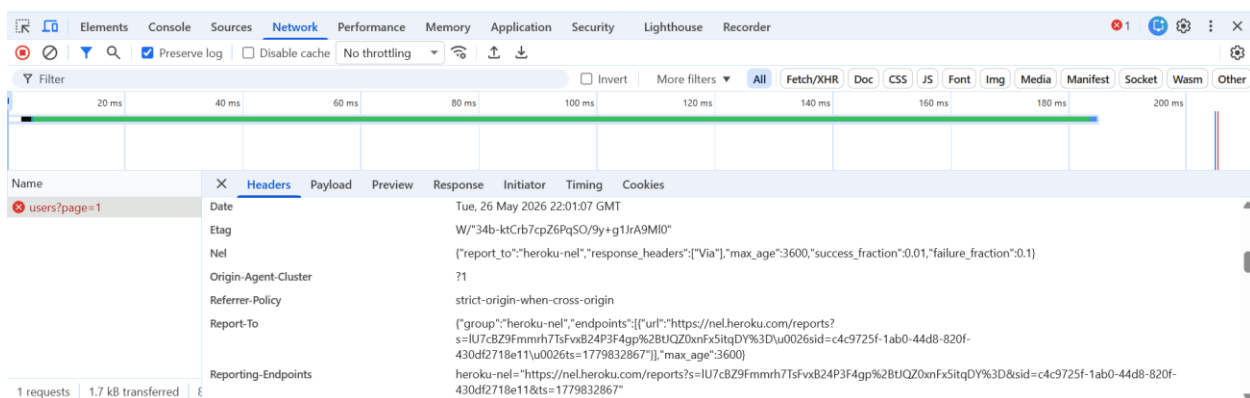


Figure 7: Additional security protections identified within the API response headers

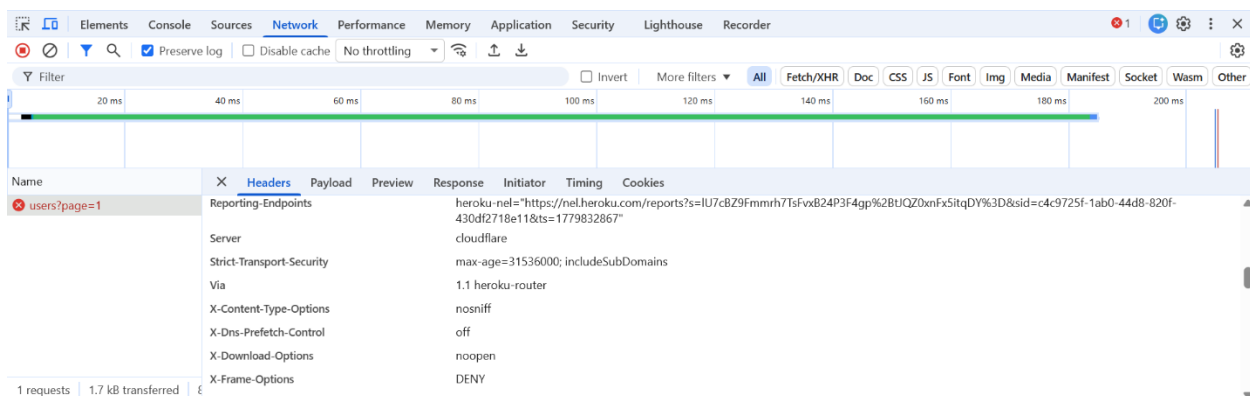


Figure 8: Authentication-related response information returned by the ReqRes API

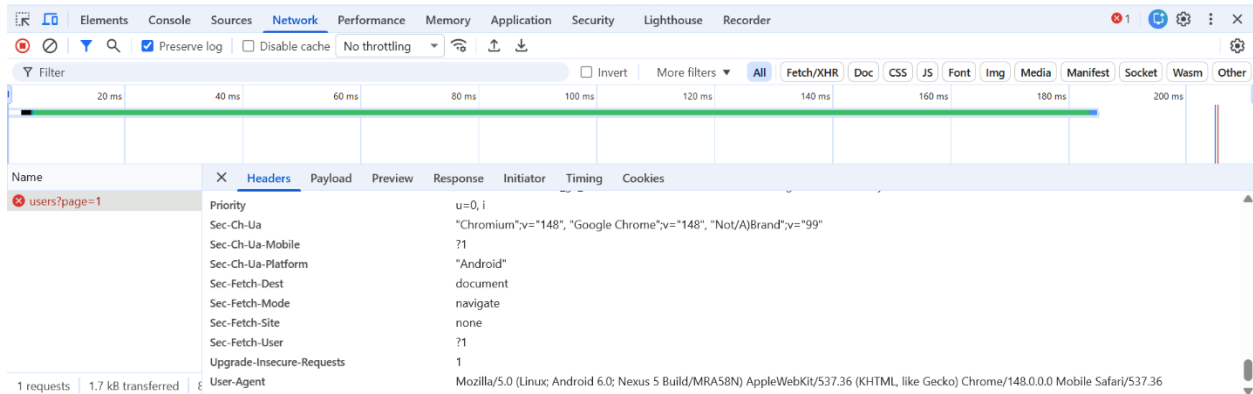


Figure 12: Final request-header inspection confirming unauthenticated API access attempt

Evidence Observed

The following security behaviours were observed during analysis:

- HTTP Status Code: 401 Unauthorised
- Error message: "missing_api_key"
- Authentication requirement: x-api-key
- Unauthorised requests were blocked successfully
- No sensitive user data was exposed without authentication

Observed response message:

json

```
{
  "error": "missing_api_key",
  "message": "The x-api-key header is required for this endpoint."
}
```

Additional response headers observed:

- strict-transport-security
- x-frame-options: DENY
- x-content-type-options: nosniff

These headers further show that security protections are enforced on the platform.

Risk Severity

Low / Informational

Business Impact

The API correctly restricts unauthenticated users from accessing protected resources. This minimises the risk of unauthorised data exposure, account misuse, and API abuse attempts.

Proper authentication enforcement is an important security control in SaaS environments because APIs frequently process sensitive customer and application data.

Remediation

No critical issue was detected in this finding. Nevertheless, the following best practices are recommended:

- Continue enforcing authentication on sensitive endpoints
- Implement secure API key rotation policies
- Apply rate limiting to reduce automated abuse attempts
- Monitor failed authentication requests for suspicious activity
- Avoid exposing excessive authentication details in error messages

Finding 2 - Information Disclosure Through Response Headers

Description

A security assessment was performed on the ReqRes API endpoint:

GET <https://reqres.in/api/users?page=1>

During response analysis, several HTTP response headers were observed that exposed backend platform details, infrastructure information, and internal service references. The assessment was performed using Insomnia and Browser Developer Tools (Chrome DevTools) to inspect HTTP response headers returned by the API.

The analysis revealed multiple headers that disclosed information related to the hosting environment, backend technologies, platform branding, upgrade references, and internal deployment messages. Although no sensitive customer data was explicitly exposed, excessive information disclosure may support attackers during reconnaissance and target profiling activities.

401 Unauthorized		995 ms	854 B	3 Minutes Ago ▼	
Preview	Headers 33	Cookies	Tests 0 / 0	→ Mock	Console
⌵					
server	cloudflare				
strict-transport-security	max-age=31536000; includeSubDomains				
via	1.1 heroku-router				
x-content-type-options	nosniff				
x-dns-prefetch-control	off				
x-download-options	noopen				
x-frame-options	DENY				
x-permitted-cross-domain-policies	none				
x-powered-by	ReqRes.in - Deploy backends in 30 seconds				
x-reqres-docs	https://reqres.in				
x-reqres-help	missing_api_key				
x-reqres-message	This API is powered by ReqRes. Deploy your own backend in 30 seconds!				
x-reqres-templates	https://app.reqres.in/templates				

Figure 13: Insomnia Response Headers Showing Security and Platform Information

401 Unauthorized		995 ms	854 B	4 Minutes Ago ▼	
Preview	Headers 33	Cookies	Tests 0 / 0	→ Mock	Console
x-download-options	noopen				
x-frame-options	DENY				
x-permitted-cross-domain-policies	none				
x-powered-by	ReqRes.in - Deploy backends in 30 seconds				
x-reqres-docs	https://reqres.in				
x-reqres-help	missing_api_key				
x-reqres-message	This API is powered by ReqRes. Deploy your own backend in 30 seconds!				
x-reqres-templates	https://app.reqres.in/templates				
x-reqres-tip	Need more than /api/users? Build custom collections - app.reqres.in/dashboard				
x-reqres-upgrade	https://app.reqres.in/upgrade				
x-request-id	0c235948-6656-a917-0652-a1ce6213c1f5				
x-robots-tag	noindex, nofollow				
x-xss-protection	0				

Figure 14: Insomnia Response Headers Showing Internal Service References

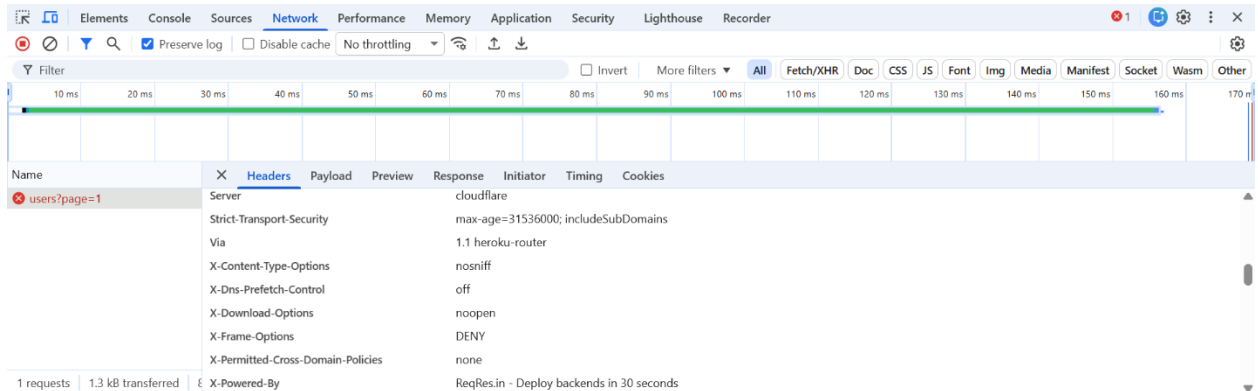


Figure 15: Browser DevTools Header Inspection

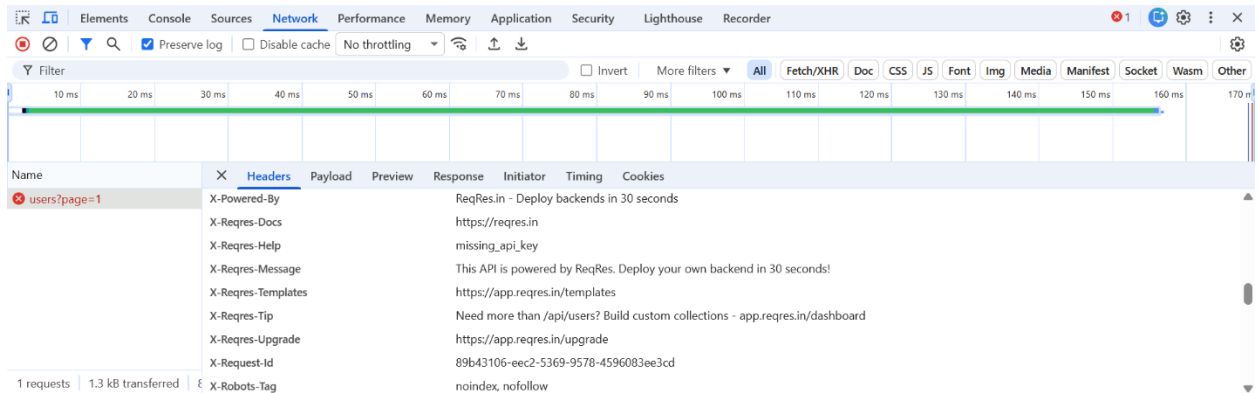


Figure 16: Browser DevTools Extended Header Analysis

Evidence Observed

The following information disclosure behaviours were identified during analysis:

- Backend technology disclosure through x-powered-by
- Hosting infrastructure disclosure through server
- Internal platform references exposed in custom headers
- Deployment and upgrade references visible in responses
- Informational service messages returned to unauthenticated users

Observed response headers included:

- server: cloudflare
- x-powered-by: ReqRes.in - Deploy backends in 30 seconds
- x-reqres-message
- x-reqres-tip
- x-reqres-upgrade
- x-reqres-docs

Risk Severity

Low/Informational

Business Impact

Exposed response headers may disclose attackers with useful reconnaissance information about the underlying infrastructure, backend technologies, and deployment environment of the API platform.

Although the disclosed information does not directly reveal sensitive customer data, unnecessary header disclosure can help attackers in identifying technologies, exploiting known vulnerabilities, and improving attack planning activities.

Remediation

The following security best practices are recommended:

- Minimise unnecessary information disclosure in HTTP response headers
- Remove or restrict non-essential custom headers
- Avoid exposing backend technology details to unauthenticated users
- Suppress internal deployment and upgrade references in production environments
- Regularly review response headers for excessive information exposure

Finding 3 - Presence of Security Headers

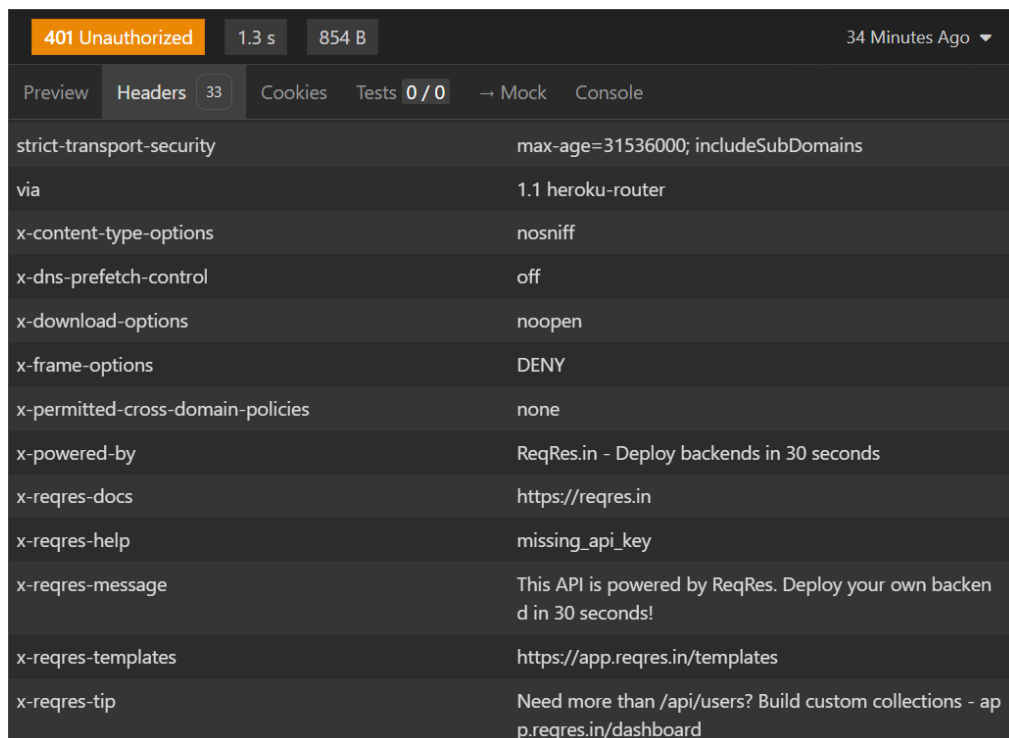
Description

A security assessment was performed on the ReqRes API endpoint:

GET <https://reqres.in/api/users?page=1>

During HTTP response analysis, multiple security-related response headers were observed within the API responses. The assessment was performed using Insomnia and Browser Developer Tools (Chrome DevTools) to inspect response headers, transport protections, and browser security configurations enforced by the platform.

The analysis demonstrated that the API implements multiple standard HTTP security headers designed to enhance browser-side protection mechanisms and reinforce overall application security. These headers assist in mitigating risks associated with insecure transport communication, clickjacking attacks, MIME-type sniffing, and cross-domain policy abuse.



The screenshot shows the Insomnia API client interface. At the top, a status bar indicates a '401 Unauthorized' response with a status of 1.3 s and a body size of 854 B, received 34 minutes ago. Below this, a tabbed interface shows 'Headers' selected, displaying 33 headers. The headers are listed in a table format with two columns: the header name and its value.

Header	Value
strict-transport-security	max-age=31536000; includeSubDomains
via	1.1 heroku-router
x-content-type-options	nosniff
x-dns-prefetch-control	off
x-download-options	noopen
x-frame-options	DENY
x-permitted-cross-domain-policies	none
x-powered-by	ReqRes.in - Deploy backends in 30 seconds
x-reqres-docs	https://reqres.in
x-reqres-help	missing_api_key
x-reqres-message	This API is powered by ReqRes. Deploy your own backend in 30 seconds!
x-reqres-templates	https://app.reqres.in/templates
x-reqres-tip	Need more than /api/users? Build custom collections - ap.p.reqres.in/dashboard

Figure 17: Security Headers Identified Using Insomnia

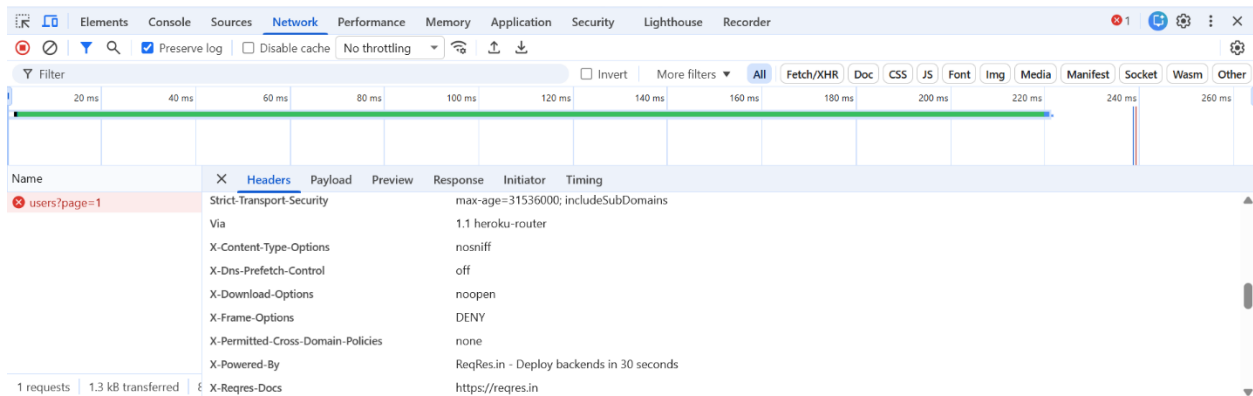


Figure 18: Security Headers Identified Using Chrome DevTools

Evidence Observed

The following security headers were identified during testing:

- strict-transport-security
- x-frame-options
- x-content-type-options
- x-permitted-cross-domain-policies
- x-download-options

Observed header values included:

- strict-transport-security: max-age=31536000; includeSubDomains
- x-frame-options: DENY
- x-content-type-options: nosniff
- x-permitted-cross-domain-policies: none

These headers show that the platform implements browser-level security protections and secure transport enforcement controls.

Risk Severity

Informational / Positive Security Observation

Business Impact

The implementation of HTTP security headers strengthens the overall security posture of the API platform and assists in reducing exposure to common browser-based attacks.

For example:

- strict-transport-security assists in enforcing secure HTTPS communication
- x-frame-options helps reduce clickjacking risks
- x-content-type-options assists in preventing MIME-type sniffing attacks
- x-permitted-cross-domain-policies limits unsafe cross-domain policy usage

Appropriate use of security headers is considered an important protective security practice for modern SaaS platforms and API-driven applications.

Remediation

The following best practices are recommended to maintain strong browser-side security protections:

- Continue enforcing HTTP Strict Transport Security (HSTS)
- Regularly review security header configurations
- Ensure security headers are consistently applied across all API endpoints
- Periodically assess header configurations against industry best practices
- Monitor infrastructure changes that may unintentionally weaken security header protections

Finding 4 - Authentication Enforcement on API Endpoint

Description

A security assessment was performed on the ReqRes API endpoint:

GET <https://reqres.in/api/users?page=1>

During testing, the API returned a 401 Unauthorised response when requests were sent without a valid API key or authentication mechanism. The assessment was carried out using Insomnia and Browser Developer Tools (Chrome DevTools) to examine authentication behaviour, request handling, response headers, and server-side access control mechanisms.

The analysis confirmed that the API applies authentication requirements before granting access to protected resources. Requests sent without valid authentication credentials were rejected by the server, demonstrating that unauthorised access controls are implemented on the endpoint.

The response also included authentication-related indicators such as:

- 401 Unauthorised
- x-reqres-help: missing_api_key

These responses indicate that the API validates authentication requirements and prevents unauthenticated requests from accessing protected API functionality.

PreviewHeaders33CookiesTests0 / 0→ MockConsole

NAME	VALUE
date	Tue, 26 May 2026 21:40:39 GMT
content-type	application/json; charset=utf-8
content-length	843
access-control-allow-origin	*
cache-control	no-store
cross-origin-opener-policy	same-origin
cross-origin-resource-policy	same-origin
etag	W/"34b-ktCrb7cpZ6PqSO/9y+g1JrA9MI0"
nel	{"report_to":"heroku-nel","response_headers":["Via"],"max_age":3600,"success_fraction":0.01,"failure_fraction":0.1}
origin-agent-cluster	?1
referrer-policy	strict-origin-when-cross-origin
report-to	{"group":"heroku-nel","endpoints":[{"url":"https://nel.heroku.com/reports?s=5D%2B9F33IFihROU9DeXs3NomhiBbjvTc%2BP3mVDIthuC4%3D\u0026sid=c4c9725f-1ab0-44d

Figure 19: Response Headers Observed During API Testing Using Insomnia

Preview

Headers

33

Cookies

Tests

0 / 0

→ Mock

Console

x-frame-options	DENY
x-permitted-cross-domain-policies	none
x-powered-by	ReqRes.in - Deploy backends in 30 seconds
x-reqres-docs	https://reqres.in
x-reqres-help	missing_api_key
x-reqres-message	This API is powered by ReqRes. Deploy your own backend in 30 seconds!
x-reqres-templates	https://app.reqres.in/templates
x-reqres-tip	Create your own backend with persistent data - app.reqres.in/dashboard
x-reqres-upgrade	https://app.reqres.in/upgrade
x-request-id	1438ed57-ad4b-a2fe-8dd6-dab4f8f8481a
x-robots-tag	noindex, nofollow
x-xss-protection	0
cf-cache-status	BYPASS

Figure 20: Additional Response Headers Identified Using Insomnia

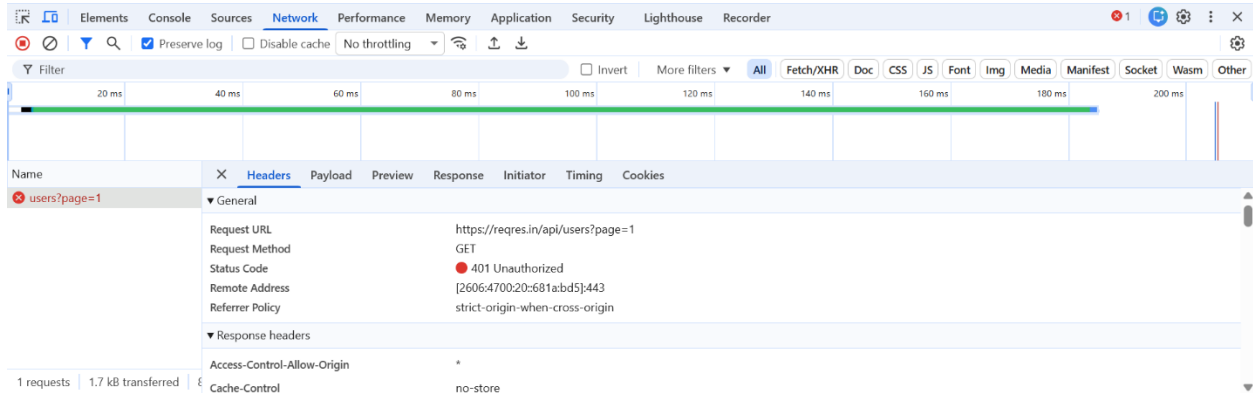


Figure 21: API Request and Response Inspection Using Chrome DevTools

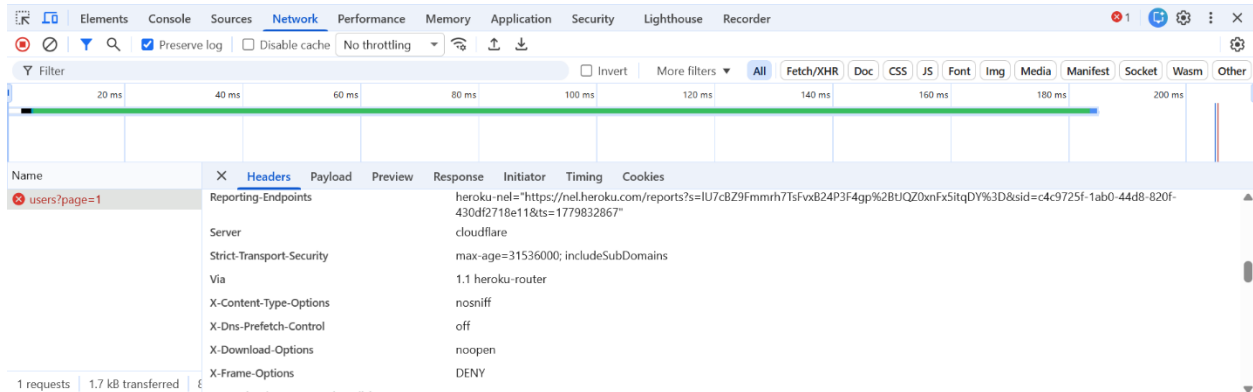


Figure 22: Response Headers Reviewed Through Chrome DevTools

Evidence Observed

The following observations were identified during testing:

- API requests without authentication returned 401 Unauthorised
- The server response included the message missing_api_key
- Access to the endpoint was denied until valid authentication credentials were provided
- Browser and API testing tools confirmed authentication enforcement behaviour

Observed response details included:

- HTTP/1.1 401 Unauthorised
- x-reques-help: missing_api_key

The analysis indicates that the platform implements authentication validation controls to prevent unauthorised API access.

Risk Severity

Low / Positive Security Observation

Business Impact

Authentication enforcement mechanisms assist in reducing the likelihood of unauthorised access to API resources and sensitive application functionality.

By requiring valid authentication credentials before processing requests, the platform enhances access control security and assists in preventing misuse of protected endpoints.

Proper authentication validation is considered an important security control for modern SaaS platforms and API-driven applications because it helps:

- reduce unauthorised data access
- protect backend services
- restrict anonymous API usage
- improve overall application security posture

Remediation

The following best practices are recommended to maintain strong API authentication security:

- Continue enforcing authentication validation on protected endpoints
- Ensure invalid or missing credentials consistently return secure error responses
- Regularly review API authentication configurations
- Apply authentication requirements across all sensitive API endpoints
- Periodically assess authentication controls against OWASP API Security best practices

Finding 5 - Cross-Origin Resource Sharing (CORS) Configuration Observation

Description

A security assessment was performed on the ReqRes API endpoint:

GET <https://reqres.in/api/users?page=1>

During HTTP response analysis, a permissive Cross-Origin Resource Sharing (CORS) configuration was identified within the API response headers. The assessment was conducted using Insomnia and Browser Developer Tools (Chrome DevTools) to inspect response headers, browser communication policies, and API access behaviour.

The analysis identified the following response header:

access-control-allow-origin: *

This configuration allows cross-origin requests from all domains using the wildcard (*) value.

CORS is a browser security mechanism that controls how web applications from different origins interact with APIs and web services. A permissive CORS policy may increase exposure if sensitive API functionality or confidential data becomes accessible through unrestricted cross-origin requests.

The observation does not confirm exploitation or unauthorised access; however, it indicates that the API permits requests from any external origin.

Preview

Headers

33

Cookies

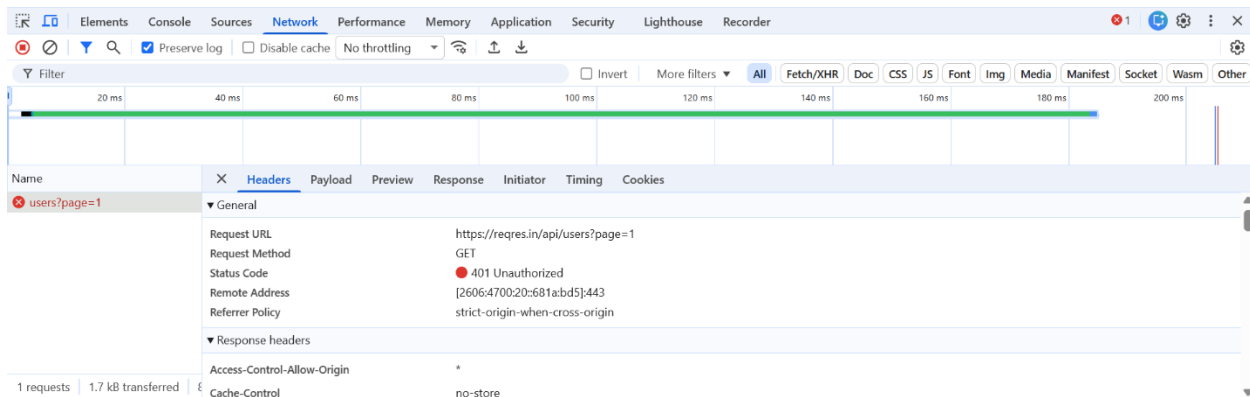
Tests

0 / 0

→ Mock

Console

NAME	VALUE
date	Tue, 26 May 2026 21:40:39 GMT
content-type	application/json; charset=utf-8
content-length	843
access-control-allow-origin	*
cache-control	no-store
cross-origin-opener-policy	same-origin
cross-origin-resource-policy	same-origin
etag	W/"34b-ktCrb7cpZ6PqSO/9y+g1JrA9MI0"
nel	{"report_to":"heroku-nel","response_headers":["Via"],"max_age":3600,"success_fraction":0.01,"failure_fraction":0.1}
origin-agent-cluster	?1
referrer-policy	strict-origin-when-cross-origin
report-to	{"group":"heroku-nel","endpoints":[{"url":"https://nel.heroku.com/reports?s=5D%2B9F33IFihROU9DeXs3NomhiBbjvTz%2BBD3mVDIthvC4%3Dw0026cid=s4s0725f1eb0_44d



Evidence Observed

The following response header was identified during testing:

access-control-allow-origin: *

Observed response details included:

access-control-allow-origin: *

The analysis indicates that the API response allows browser-based requests from all origins without applying domain-specific restrictions.

Risk Severity Level

Low / Informational

Business Impact

Permissive CORS configurations may increase browser-based exposure by allowing external websites or applications to interact with the API from different origins.

If sensitive functionality or confidential data is exposed through unrestricted CORS policies, attackers may attempt to abuse browser-based interactions through malicious or untrusted web applications.

Potential security concerns associated with overly permissive CORS configurations may include:

- increased cross-origin exposure
- unintended browser-based API access
- misuse by untrusted third-party applications

- broader attack surface for web-based integrations

The actual security impact depends on the sensitivity of the API endpoints and the authentication mechanisms implemented by the platform.

Remediation

The following best practices are recommended to maintain secure CORS configurations:

- Restrict allowed origins to trusted domains where possible
- Avoid using wildcard (*) origin policies for sensitive API functionality
- Regularly review CORS configurations across API endpoints
- Apply least-privilege principles to cross-origin access policies
- Periodically assess browser-based API exposure against security best practices

4. API Security Assessment Findings Overview

Sr.	Finding	Category	Risk	Evidence
1	Unauthorised Request Blocking	Authentication Control	Low / Informational	HTTP 401 Unauthorized
2	Information Disclosure Through Response Headers	Information Disclosure	Low / Informational	Platform details exposed in headers
3	Presence of Security Headers	Security Configuration	Informational / Positive	HSTS, X-Frame-Options, Nosniff
4	Authentication Enforcement on API Endpoint	Access Control	Low / Positive	missing_api_key response
5	CORS Configuration Observation	CORS Configuration	Low / Informational	access-control-allow-origin: *

5. Remediation Recommendations

Based on the observations identified during the API security assessment, the following recommendations are provided to further strengthen the security posture of the platform:

1. Continue enforcing authentication requirements on all sensitive API endpoints to prevent unauthorised access.
2. Regularly review API response headers and minimise unnecessary information disclosure that could assist attackers during reconnaissance activities.
3. Maintain and periodically validate HTTP security header configurations to ensure consistent browser-side protection across all API services.
4. Implement secure API key management practices, including key rotation, monitoring, and access control reviews.
5. Review Cross-Origin Resource Sharing (CORS) configurations and restrict allowed origins where business requirements permit.
6. Apply the principle of least privilege to API access controls and authentication mechanisms.
7. Continuously monitor API activity and authentication failures to identify suspicious behaviour and potential abuse attempts.
8. Conduct periodic API security assessments against industry standards such as the OWASP API Security Top 10 to identify emerging security risks.
9. Ensure all API communications continue to use secure transport protections and encrypted connections.
10. Establish a regular security review process for API configurations, authentication controls, and exposed services to maintain a strong security posture.

6. Conclusion

This API Security Risk Analysis evaluated the publicly accessible ReqRes API using a passive, read-only testing methodology. The assessment focused on authentication controls, response headers, security configurations, and browser-based access mechanisms without attempting exploitation or unauthorised access.

The analysis identified five key findings, comprising authentication enforcement mechanisms, information disclosure through response headers, implementation of HTTP security headers, access control validation, and a permissive Cross-Origin Resource Sharing (CORS) configuration. No severe vulnerabilities or evidence of unauthorised data exposure were detected during the assessment. Authentication controls were found to function effectively by denying requests that lacked valid API credentials, while multiple security headers were present to support browser-side protection mechanisms.

Overall, the API exhibited a generally strong security posture and effective implementation of several fundamental security controls. The observations identified primarily relate to security hardening opportunities and configuration improvements rather than exploitable weaknesses. Continued monitoring, regular security reviews, and compliance with OWASP API Security best practices will assist in maintaining a secure and resilient API environment.